Simulando Gerenciamento de Restaurante

Mário Leite

Uma das melhores formas de aprender "Programação Orientada a Objetos" é modelar situações do mundo real; um restaurante é um exemplo perfeito: ele tem nome, tipo de cozinha, recebe pedidos, soma valores. e ao final fecha a conta. Nesta postagem vamos construir um programa simples em Python que simula o funcionamento básico de um restaurante, utilizando uma classe e alguns métodos que representam ações reais do dia a dia.

1. A ideia do modelo

O restaurante será representado por uma classe chamada **ClsRestaurante**. Cada objeto dessa classe será um restaurante específico, contendo:

- nome
- tipo de cozinha
- número de pedidos
- Total gasto pelo cliente

Ou seja, cada instância é um restaurante "de verdade" dentro do programa.

2. Estrutura da classe

```
class ClsRestaurante:
def __init__(self, nome_rest, tipo_cozinha):
    self.nome_rest = nome_rest
    self.tipo_cozinha = tipo_cozinha
    self.num_pedidos = 0
    self.tot_desp = 0
```

O método **__init__** é o **construtor**. Ele define o estado inicial do restaurante:

- começa com zero pedidos
- começa com zero de despesas

Isso representa o restaurante antes de atender qualquer cliente.

3. Descrevendo o restaurante

```
def DescreverRestaurante(self):
    print(f"Nome do restaurante: '{self.nome_rest}'")
    print(f"Tipo de cozinha: '{self.tipo_cozinha}'")
```

Este método apenas exibe as informações básicas; é como se fosse a "placa" na frente do restaurante.

4. Abrindo o restaurante

```
def AbrirRestaurante(self):
    print()
    print(f"O restaurante '{self.nome_rest}' está aberto!")
```

Aqui temos uma ação simbólica: o restaurante passa a operar; do ponto de vista didático, isso mostra como métodos podem representar **eventos do sistema**.

5. Atendendo pedidos

```
def AtenderPedido(self, pedido, valor):
    self.num_pedidos += 1
    self.tot_desp += valor
    print(f"Pedido atendido: '{pedido}'")
```

Este é o método mais importante do programa.

A cada pedido:

- o contador de pedidos aumenta
- o valor é somado ao total

Ou seja, o objeto muda de estado internamente.

6. Fechando a conta

```
def FecharConta(self):
    print(f"Total de pedidos atendidos: {self.num_pedidos}")
    print(f"Total das despesas: R$ {self.tot_desp:.2f}")
```

Aqui o sistema apresenta o resultado final da operação do restaurante.

<u>Programa principal (uso real)</u>

```
Restaurante = ClsRestaurante("La Bella Italia", "Italiana")
Restaurante.DescreverRestaurante()
Restaurante.AbrirRestaurante()
Restaurante.AtenderPedido("Pizza Margherita", 85)
Restaurante.AtenderPedido("Lasanha à Bolonhesa", 80)
Restaurante.AtenderPedido("Tiramisu", 75)
Restaurante.FecharConta()
```

Observe como o código fica **legível e natural**, quase como uma narrativa:

- cria o restaurante
- abre
- atende pedidos
- fecha a conta

Isto é uma das maiores vantagens da orientação a objetos.

Conceitos importantes envolvidos

Esse programa simples já demonstra vários conceitos fundamentais:

- **Classe**: modelo genérico do restaurante
- **Objeto**: restaurante específico ("La Bella Italia")
- **Atributos**: estado interno (pedidos e despesas)
- **Métodos**: comportamentos (atender, abrir, fechar)
- **Encapsulamento**: os dados ficam protegidos dentro do objeto

Vantagens desse modelo

Mesmo sendo simples, esse modelo é poderoso porque:

- evita variáveis globais
- organiza a lógica em torno de um conceito central
- facilita manutenção
- permite criar vários restaurantes diferentes
- aproxima o código da realidade

Este tipo de exemplo mostra que a Programação Orientada a Objetos não é sobre "moda" ou "estilo", mas sobre modelar o mundo real de forma organizada e intuitiva. Observe o programa **"MovimentoEmRestaurante"** e a saída na **figura 1**.

```python
'''
MovimentoEmRestaurante.py
-------------------------------------------------------------------------------
Faz movimentações dos pedidos de um cliente em um restaurante usando classe.
-------------------------------------------------------------------------------
'''
class ClsRestaurante:
    def __init__(self, nome_rest, tipo_cozinha):
        self.nome_rest = nome_rest
        self.tipo_cozinha = tipo_cozinha
        self.num_pedidos = 0
        self.tot_desp = 0

    def DescreverRestaurante(self):
        print(f"Nome do restaurante: '{self.nome_rest}'")
        print(f"Tipo de cozinha: '{self.tipo_cozinha}'")

    def AbrirRestaurante(self):
        print()
        print(f"O restaurante '{self.nome_rest}' está aberto!")

    def AtenderPedido(self, pedido, valor):
        self.num_pedidos += 1
        self.tot_desp += valor
        print(f"Pedido atendido: '{pedido}'")

    def FecharConta(self):
        print(f"Total de pedidos atendidos: {self.num_pedidos}")
        print(f"Total das despesas: R$ {self.tot_desp:.2f}")

#===============================================================================
#Programa principal
Restaurante = ClsRestaurante("La Bella Italia", "Italiana")
#Chama os métodos
Restaurante.DescreverRestaurante()
Restaurante.AbrirRestaurante()

#Atendendo pedidos dados
Restaurante.AtenderPedido("Pizza Margherita", 85)
Restaurante.AtenderPedido("Lasanha à Bolonhesa", 80)
Restaurante.AtenderPedido("Tiramisu", 75)

Restaurante.FecharConta() #fechando a conta
#Fim do programa "MovimentoEmRestaurante" ---------------------------------
```

```
Run:  MovimentoRestaurante
C:\Users\Usuario\AppData\Local\Python\python310\Scripts\python.exe "D:\Livros\Livro11
Nome do restaurante: 'La Bella Italia'
Tipo de cozinha: 'Italiana'

O restaurante 'La Bella Italia' está aberto!
Pedido atendido: 'Pizza Margherita'
Pedido atendido: 'Lasanha à Bolonhesa'
Pedido atendido: 'Tiramisu'
Total de pedidos atendidos: 3
Total das despesas: R$ 240.00

Process finished with exit code 0
```

Figura 1 - Saída do programa "MovimentoEmRestaurante"